Sistemas de Tiempo Real — Práctica 3

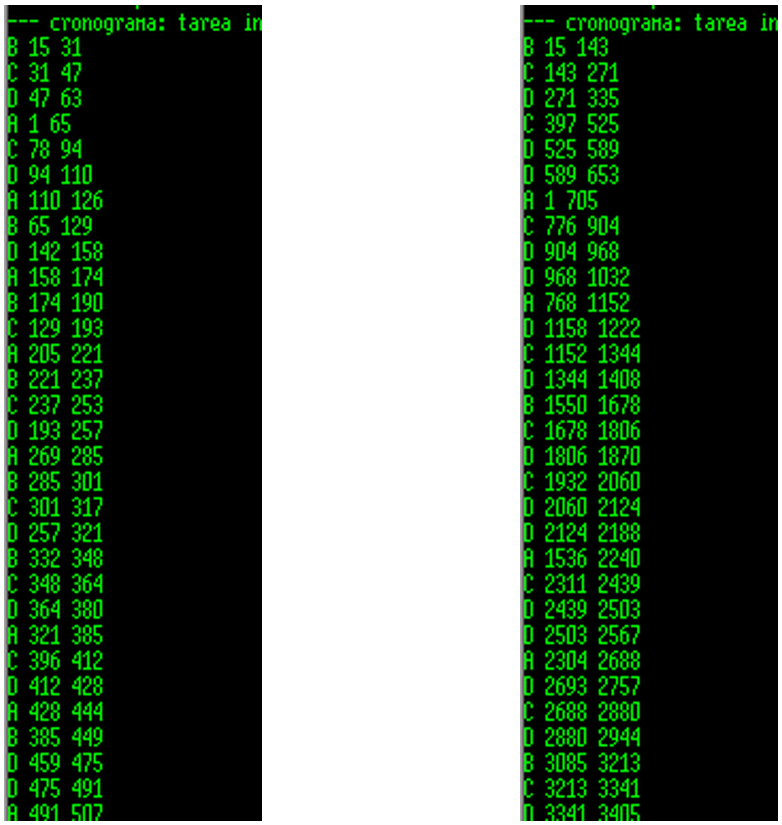
Ejercicio 5: Planificación FIFO

Ingeniería en Computación

Método

Se mantienen las cuatro tareas del Ejercicio 1 con igual prioridad y se deshabilita el reparto por cuantos. En este banco, por lo explicado en el informe del Ejercicio 4, el cuanto no lo produce la interrupción del tick sino la propia tarea con `taskYIELD()` al cerrar cada tramo de 16 ms; pasar a FIFO consiste en no ceder, de modo que la ráfaga se ejecuta completa. Es un único cambio de una línea y no requiere tocar la biblioteca.

Para comparar los cronogramas sin depender de la configuración del analizador lógico, el firmware registra en memoria los tramos de ejecución (cuándo cada tarea toma y suelta el procesador) y los imprime por la terminal con los instantes en milisegundos.



Round Robin FIFO

Figura 1: Cronogramas de ejecución, en milisegundos: tarea, instante de inicio e instante de fin de cada tramo.

a) Comparación cualitativa de los cronogramas

La diferencia es inmediata. Bajo Round Robin los tramos duran **16 ms** y las cuatro tareas se turnan sin parar: `B 15 31`, `C 31 47`, `D 47 63`, `C 78 94`, `D 94 110`, `A 110 126`. En el mismo intervalo hay más de treinta cambios de contexto.

Bajo FIFO los tramos duran lo que dura el cómputo entero: `B 15 143` son 128 ms, `C 143 271` otros 128 y `D 271 335` son 64. En el mismo intervalo hay seis tramos en lugar de treinta.

Round Robin reparte el procesador en el tiempo; FIFO lo entrega de a uno y completo.

b) Atomicidad y expropiación

Las ráfagas son atómicas. Se verifica restando las dos columnas del cronograma: todos los tramos de A, B y C miden **128 ms** y los de D miden **64 ms**, que son exactamente sus tiempos de cómputo ($C = 2$ y $C = 1$ unidades, con 1 unidad = 64 ms). Ninguna ráfaga aparece partida.

¿Sigue siendo expropiativo? Sí. Entre prioridades distintas la expropiación es la del sistema operativo: la tarea de fondo, que corre en la tarea Idle, es desalojada en cuanto se libera cualquier periódica. Lo que desaparece no es la expropiación sino el reparto por cuantos entre tareas de igual prioridad.

¿En qué situación podría producirse igualmente una expropiación? El cronograma la muestra. Las líneas de A no son tramos de 128 ms sino ventanas largas: `A 1 705`, `A 768 1152`, `A 1536 2240`. Lo mismo le

ocurre a veces a C: `C 1152 1344`, `C 2688 2880`. Esas tareas arrancaron su ráfaga y fueron desalojadas antes de terminarla.

La causa es que `configUSE_TIME_SLICING` sigue valiendo 1 en la biblioteca: el reparto por tiempo del núcleo no se deshabilitó, solo se dejó de invocar `taskYIELD()`. El despachador de activaciones suspende y reanuda el planificador, y al reanudarlo FreeRTOS procesa los ticks que quedaron pendientes; si en ese momento hay otra tarea lista de igual prioridad, `xTaskIncrementTick()` solicita rotación y `xTaskResumeAll()` cede el procesador. La expropiación ocurre entonces **en los bordes donde se reanuda el planificador**, aunque la tarea nunca ceda explícitamente.

c) Orden de atención de tareas activadas en el mismo instante

Las cuatro tareas se crean en el orden A, B, C, D y el despachador las libera en ese mismo orden, con el planificador suspendido, de modo que quedan listas en el mismo instante. El orden observado de atención es, sin embargo,

$$\mathbf{B} \rightarrow \mathbf{C} \rightarrow \mathbf{D} \rightarrow \mathbf{A}$$

B toma el procesador a los 15 ms, C a los 143, D a los 271, y A recién completa a los 705. **No coincide con el orden de llegada.**

La explicación está en cómo FreeRTOS maneja la lista de listos. `prvAddTaskToReadyList` llama a `vListInsertEnd`, que inserta el elemento *delante del índice actual* de una lista circular; y la selección, `listGET_OWNER_OF_NEXT_ENTRY`, **avanza ese índice** antes de devolver el dueño. El índice no se reinicia entre selecciones: es estado que persiste de la vuelta anterior.

En consecuencia el recorrido es cíclico sobre la lista y arranca desde donde había quedado, no desde el primero que llegó. La lista de listos de FreeRTOS no es una cola FIFO por orden de llegada, aunque la política de ejecución sí sea no expropiativa. Es una distinción que conviene no confundir: lo *FIFO* del Ejercicio 5 es que la ráfaga no se interrumpe, no que se atienda en orden de llegada.

d) Resultados comparados

	Respuesta en el instante crítico		Plazos perdidos			
Tarea	Round Robin	FIFO	Round Robin	FIFO	T (ms)	
A	507 ms	**705 ms**	0	0	768	empeora
B	523 ms	**143 ms**	0	0	1536	mejora
C	529 ms	**271 ms**	1 por hiperper.	**0**	384	mejora
D	257 ms	**335 ms**	1 por hiperper.	**3 por hiperper.**	192	empeora

Mejoran B y C, que son las atendidas primero: B pasa de 523 a 143 ms, una reducción del 73 %, y C de 529 a 271. C además deja de perder plazos por completo.

Empeoran A y D. A pasa de 507 a 705 ms porque queda última en el recorrido de la lista y la pasan por encima las tareas que se reactivan mientras espera. D pasa de 257 a 335 ms y, sobre todo, triplica sus plazos perdidos: de uno por hiperperíodo a tres, porque con ráfagas atómicas puede quedar bloqueada hasta 128 ms detrás de una tarea cuyo plazo no corre riesgo.

El total de trabajo no cambió: bajo ambas políticas el conjunto completo de primeras ráfagas termina cerca de los 450 ms. Lo que cambia es *a quién* se le cobra la espera.

e) Conclusión

Ninguna de las dos políticas garantiza el cumplimiento de plazos. Round Robin reparte el perjuicio entre dos tareas, C y D, que pierden un plazo por hiperperíodo cada una. FIFO lo concentra en una sola: C queda en cero pero D pasa a perder el 37 % de sus activaciones.

La información que ambos planificadores ignoran es la misma: **el período y el plazo de cada tarea**. Round Robin solo conoce el cuanto; FIFO solo conoce el orden de la lista de listos. Ninguno de los dos recibe el dato de que D tiene que terminar cada 192 ms mientras B dispone de 1536, de modo que tratan igual a tareas con urgencias que difieren en un factor de ocho.

De ahí que la respuesta no esté en elegir mejor entre estas dos, sino en cambiar la clase de política: asignar prioridades en función del período, como hace Rate Monotonic, o en función del plazo más próximo, como hace Earliest Deadline First. Con el conjunto de este trabajo, $U = 0{,}9167 < 1$, una planificación por prioridades según el período cumpliría todos los plazos sobre el mismo hardware y el mismo código de las tareas; lo único que cambia es qué sabe el planificador.